

1h30 - Sans documents excepté une copie double resto-verso avec notes manuscrites

La clarté de vos réponses sera prise en compte. Ne pas écrire au crayon.

N'oubliez pas de commenter vos programmes

Vous aurez besoin de 2 copies : une par partie

Attention à bien conserver l'indentation tout au long de votre copie. Vous pouvez écrire en commentaire dans votre programme ou dans la marge le n° de la question.

Partie 1 : Base de données (10 points) -> copie n°1

1. Compagnie aérienne (5 points) :

Pour gérer ses vols, une compagnie aérienne dispose des éléments suivants :

Un vol est constitué d'un numéro de vol (ex : AF303), à chaque vol est associé un avion ainsi qu'un équipage et des passagers.

Un avion comprend le type d'avion (ex : Airbus A350), le nombre de passagers et le nombre de personnels navigants.

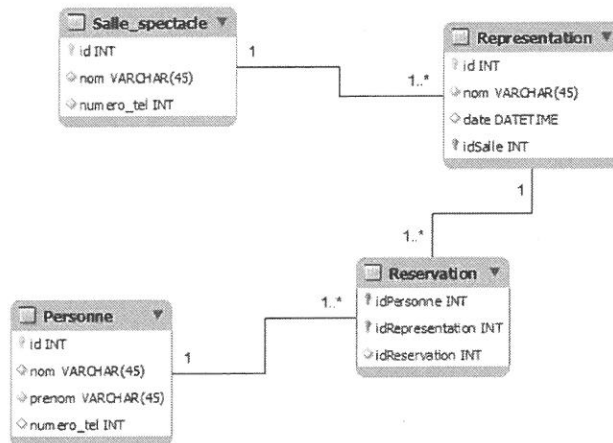
Un personnel navigant a un nom, un prénom, un numéro de téléphone.

Un passager a un nom, prénom.

- 1.1. Identifiez les différentes tables ainsi que les clés primaires.
- 1.2. Identifiez les relations entre les différents éléments ainsi que leur type et cardinalité.
- 1.3. Dessiner le diagramme de la base de données.

2. Interrogation BDD (5 points) :

Soit le diagramme suivant :



Les clés roses représentent des clés étrangères.

Écrire les requêtes SQL permettant de :

- 2.1. Lister les dates des différentes représentations de « Tartuffe ».
- 2.2. Lister toutes les salles dans lesquelles ont lieu le représentation de « Tartuffe ».
- 2.3. Récupérer le numéro de téléphone de toutes les personnes ayant réservé un spectacle dans le théâtre « Molière ».

Partie 2 : Tkinter et Numpy (10 points) -> copie n°2

3. Fenêtre (7 points) :

Soit le programme suivant :

```

import tkinter as tk
import random as rd

class AppliCanevas(tk.Tk):
    def __init__(self):
        tk.Tk.__init__(self)
        self.size = 500
        geometrie = f"{100}x{100}"
        self.geometry(geometrie)
        self.creer_widgets()
  
```

```

def creer_widgets(self):
    self.canv = tk.Canvas(self, bg="light gray", height=self.size, width=self.size)
    self.canv.pack()
    self.bouton_cercles = tk.Button(self, text="Cercles", command=self.dessine_cercles)
    self.bouton_cercles.pack()
    self.bouton_lignes = tk.Button(self, text="Lignes", command=self.dessine_lignes)
    self.bouton_lignes.pack()
    self.bouton_quitter = tk.Button(self, text="Quitter", command=self.quit)
    self.bouton_quitter.pack()

def rd_couleur(self):
    return rd.choice(("black", "red", "green", "blue", "yellow", "magenta",
                    "cyan", "white", "purple"))

def dessine_cercles(self):
    n = rd.randint(1, 15)
    for i in range(n):
        x, y = [rd.randint(1, self.size) for j in range(2)]
        diameter = rd.randint(1, 50)
        self.canv.create_oval(x, y, x+diameter, y+diameter, fill=self.rd_couleur())

def dessine_lignes(self):
    n = rd.randint(1, 10)
    for i in range(n):
        x, y, x2, y2 = [rd.randint(1, self.size) for j in range(4)]
        self.canv.create_line(x, y, x2, y2, fill=self.rd_couleur())

if __name__ == "__main__":
    app = AppliCanevas()
    app.title("Final INF2")
    app.mainloop()

```

- 3.1. Dessiner sur votre copie la fenêtre créée par ce programme (rd.choice() renvoie une chaîne au hasard du tuple passé en argument).
- 3.2. Au lancement du programme, à quel endroit de l'écran se situe la fenêtre ?
- 3.3. Décrire en quelques lignes ce qu'il se passe lorsqu'on appuie sur les boutons « cercles » et « lignes ».
- 3.4. Modifier la fonction creer_widgets() pour que ces deux boutons se situent sur une colonne à droite du canevas et que le bouton « quitter » soit en dessous du canevas et aligné à droite avec les autres boutons.
- 3.5. Quel type de widgets faudrait-il rajouter pour afficher, sous le canevas, le nombre total de cercles et le nombre total de lignes dessinés dans le canevas ? Modifier le code pour intégrer et gérer correctement l'affichage de ces widgets.

4. Utilisation de numpy (3 points) :

L'objectif de cet exercice est de tester une méthode d'approximation de fonction à l'aide de la librairie numpy. Dans cet exercice, peu importe la méthode, on s'attachera à générer les différentes valeurs à afficher.

À partir d'une liste de ses points (x_i, y_i) , on cherche à approximer une fonction en supposant qu'elle est de la forme :

$$y_i = c_1 e^{-x_i} + c_2 x_i$$

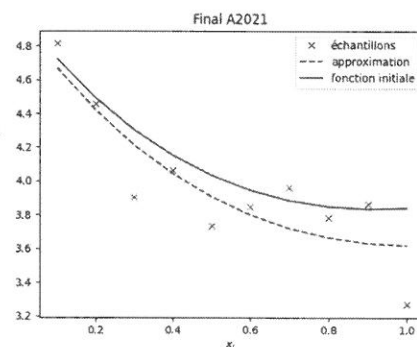
Il s'agit donc d'estimer les coefficients c_1 et c_2 de la fonction qui se rapproche le plus de ces échantillons.

Dans la suite de l'exercice, on génèrera plusieurs points d'une fonction initiale (recherchée). On ajoutera ensuite du bruit sur ces points. Enfin, on cherchera une approximation de cette fonction. On affichera alors la fonction recherchée, les points générés avec du bruit et la fonction approximée comme indiqué ci-dessous. x , y_{init} , y , y_{res} sont des ndarray

```

plt.plot(x, yinit, color='r', label="fonction initiale")
plt.plot(x, y, "x", label="échantillons")
plt.plot(x, yres, color='g', linestyle='dashed', label="approxima")
plt.xlabel('$x_i$')
plt.title('Final A2021')
plt.legend()
plt.show()

```



- 4.1. Écrire le code python pour générer les ndarray x , y_{init} , y , y_{res} sachant que :

- On définit 10 points de la fonction recherchée (x_i, y_{init_i}) avec $c_1 = 5$, $c_2 = 2$ et x_i compris entre 0.1 et 1, avec un pas de 0.1.
- Pour obtenir les y_i , ajoutez un bruit aléatoire sur chacun des y_{init_i} en utilisant la formule suivante :

$$y_i = y_{init_i} + 0.05 * \max(y_{init_i}) * a_i$$
(a_i : valeur aléatoire pour l'échantillon i)
- Pour calculer les y_{res_i} , on suppose qu'une solution d'approximation est obtenue de la manière suivante :

$$c1_res, c2_res = np.linalg.methode_approx(x, y)$$

- 4.2. Comment faire pour obtenir une meilleure précision dans l'affichage des courbes verte et rouge ?